University of Ottawa
CSI 2110 - Data Structures and Algorithms
Laboratory 3
Priority Queues, Heaps, Maps, Hash Tables, Binary Search Trees, and Balanced Search Tree Concepts
Professor Mohamed Ali Ibrahim, PhD, PEng
Term Spring/Summer 2026

Lab Statement Available	Monday, June 1, 2026
Lab Period	Monday, June 1, 2026 to Saturday, June 27, 2026
First Lab Session	Friday, June 5, 2026
Submission	Brightspace, by Saturday, June 27, 2026, 11:59 PM
Programming Language	Java

1. Laboratory Overview

The purpose of this laboratory is to help students implement and compare non-linear and associative data structures used in data structures and algorithms. This lab is aligned with the topics covered in Chapter 9, Chapter 10, and Chapter 11 of the course textbook. The focus is on priority queues, heap-based priority queues, maps, hash tables, hash functions, collision handling, load factors, and binary search trees.

- Priority queue ADT and priority-based processing
- Heap-based priority queue implementation
- Maps and key-value associations
- Hash tables, hash functions, collision handling, and load factors
- Separate chaining or linear probing for collision resolution
- Balanced search trees: AVL trees and red-black tree concepts
- Binary search tree insertion, search, traversal, and deletion
- Java implementation, testing, complexity analysis, and reporting

2. Topics Covered in Lab 3

Chapter Area	Topics Included in Lab 3
Chapter 9 - Priority Queues and Heaps	Priority queue ADT; entries and keys; heap-order property; complete binary tree; heap insertion; heap removal; heap construction; priority-based scheduling.
Chapter 10 - Maps and Hash Tables	Map ADT; key-value pairs; hash functions; compression functions; hash table implementation; collision handling; separate chaining; open addressing; linear probing; load factors.
Chapter 11 - Binary Search Trees and Balanced Search Trees	Binary search tree property; insertion; search; in-order traversal; deletion cases; tree height; balanced search tree motivation; AVL tree balance factors and rotations; red-black tree properties and conceptual comparison.

3. Learning Objectives

- Implement a generic priority queue using entries with keys and values.
- Implement a heap-based priority queue and explain the heap-order and complete-binary-tree properties.
- Use a priority queue to process tasks according to their priority and insertion time when appropriate.
- Implement a map ADT for storing and retrieving key-value pairs.
- Implement a hash table with a clear collision-handling strategy.

- Analyze the effect of the load factor on hash-table performance.
- Explain the purpose of balanced search trees and compare AVL trees and red-black trees conceptually with ordinary binary search trees.
- Implement a binary search tree and support insertion, search, traversal, and deletion.
- Compare custom implementations with Java Collections Framework classes.
- Analyze the time complexity of the required operations using Big-O notation.
- Prepare a short technical report explaining design choices, tests, and complexity trade-offs.

4. Problem Description

Students will design and implement a small Campus Task Scheduler and Directory System. The system models a simplified academic support platform where tasks are prioritized, users and tasks are stored by keys, records are retrieved through hash tables, and searchable records are maintained in a binary search tree. Students will also study how balanced search trees, including AVL trees and red-black trees, address the worst-case height problem of ordinary binary search trees.

The lab is divided into required parts. Each part may be implemented in a separate class, but all components must be demonstrated through a complete Main.java test program. The goal is not only to make the code work, but also to explain the design decisions and the efficiency of each operation.

5. Required Data Models

Students must define simple data classes similar to the following. Additional fields may be added if they are clearly explained in the report.

```
public class Task {
    private int id;
    private String title;
    private String requester;
    private int priority;      // smaller number may mean higher priority
    private String status;

    // constructors, getters, setters, and toString()
}

public class StudentRecord {
    private int studentNumber;
    private String name;
    private String program;
    private String email;

    // constructors, getters, setters, and toString()
}
```

6. Part A - Priority Queue ADT

Create a generic priority queue interface using entries with keys and values.

```
public interface PriorityQueue<K, V> {
    int size();
    boolean isEmpty();
    Entry<K, V> insert(K key, V value);
    Entry<K, V> min();
    Entry<K, V> removeMin();
}
```

Requirements

- Define an Entry<K,V> interface or class to store a key and a value.
- Implement an unsorted-list priority queue or a sorted-list priority queue.
- Use Task objects as values and integer priorities as keys.
- Demonstrate insert, min, removeMin, size, and isEmpty.
- Explain the difference between an unsorted and a sorted priority queue in terms of insertion and removal cost.

Expected behavior example:

```
insert Task #201 with priority 3
insert Task #202 with priority 1
insert Task #203 with priority 2
removeMin() -> Task #202 is processed first
```

7. Part B - Heap-Based Priority Queue

Implement a heap-based priority queue using an array or ArrayList representation of a complete binary tree.

Requirements

- Implement HeapPriorityQueue<K,V>.
- Use an array-based representation where the parent and child indices can be computed.
- Implement upheap and downheap operations.
- Demonstrate heap insertion and removeMin with at least 8 tasks.
- Print or display the internal heap array after selected insertions and removals.
- Explain why the heap-based implementation is efficient for priority-based scheduling.

8. Part C - Map ADT

Create and implement a simplified map structure for storing key-value pairs. The key should be unique.

```
public interface Map<K, V> {
    int size();
    boolean isEmpty();
    V get(K key);
    V put(K key, V value);
    V remove(K key);
    Iterable<K> keySet();
    Iterable<V> values();
    Iterable<Entry<K, V>> entrySet();
}
```

Requirements

- Implement an UnsortedTableMap<K,V> or a simple array/list-based map.
- Store StudentRecord objects using studentNumber as the key.
- Demonstrate insertion, replacement, search, removal, and iteration over entries.
- Clearly explain what should happen when put is called with an existing key.

9. Part D - Hash Table Implementation

Implement a hash table for efficient storage and retrieval of records.

Requirements

- Implement a hash table using either separate chaining or linear probing.

- Use a clear hash function and compression function.
- Demonstrate at least one collision and show how it is handled.
- Compute and report the load factor after several insertions.
- Support get, put, and remove operations.
- Test edge cases such as removing a missing key and inserting a duplicate key.

Students choosing linear probing must clearly handle deleted entries. Students choosing separate chaining must clearly show the bucket structure used to store collisions.

10. Part E - Binary Search Trees and Balanced Search Tree Concepts

Implement a binary search tree for storing and searching records by key.

Required Operation Description

Required Operation	Description
insert(key, value)	Insert a new key-value pair or update an existing key.
search(key)	Return the value associated with the key, or null if the key is not found.
remove(key)	Remove a node using the appropriate BST deletion case.
inOrderTraversal()	Return or print keys in sorted order.
preOrderTraversal()	Return or print a pre-order traversal.
postOrderTraversal()	Return or print a post-order traversal.
height()	Return the height of the tree.

Students must demonstrate deletion of a leaf node, deletion of a node with one child, and deletion of a node with two children.

Balanced search tree concept requirement: A full AVL tree or red-black tree implementation is not required unless explicitly chosen as an extension. However, the report must include a conceptual explanation of why balanced search trees are useful.

Students must include a small discussion covering AVL tree balance factors, the idea of rotations, the main red-black tree properties, and how balanced trees improve the worst-case search time compared with an unbalanced binary search tree.

11. Part F - Java Collections Framework Comparison

Students must compare their custom implementations with equivalent Java Collections Framework classes.

Custom Structure	Suggested Java Class for Comparison
Custom PriorityQueue implementation	java.util.PriorityQueue
Custom Map implementation	HashMap or TreeMap
Custom HashTable implementation	HashMap
Custom BinarySearchTree implementation	TreeMap or TreeSet as an approximate comparison
Balanced search tree concepts	TreeMap or TreeSet as examples of library structures based on balanced search trees

In the report, students must explain at least two similarities and two differences between their custom implementations and the selected Java library classes. The comparison should focus on behavior, efficiency, safety, and ease of use.

12. Part G - Complexity Analysis

Students must complete a written complexity analysis section in the report. The analysis must include Big-O notation and short justifications.

Required Complexity Table

Operation	Unsorted Priority	Heap Priority Queue	Unsorted Map	Hash Table	Binary Search Tree

	Queue				
insert / put					
min					
removeMin					
get / search					
remove					
keySet / traversal					
height					
worst-case note					

Required Written Questions

- Explain why a heap-based priority queue can support efficient removeMin operations.
- Explain how a hash function and a compression function work together.
- Explain the difference between separate chaining and linear probing.
- Explain how the load factor can affect hash-table performance.
- Explain why an in-order traversal of a binary search tree visits keys in sorted order.
- Explain why AVL trees and red-black trees are used to keep search-tree height balanced.
- Compare the best-case and worst-case height of a binary search tree and explain how this affects search time.
- Compare AVL trees and red-black trees at a conceptual level, including balancing strategy and practical use cases.

13. Testing Requirements

Students must create a Main.java file that demonstrates the correctness of all required implementations and exercises. The test program must include:

- Creating at least 8 Task objects and at least 8 StudentRecord objects.
- Testing priority queue insert, min, removeMin, size, and isEmpty.
- Testing heap-based priority queue upheap and downheap behavior through insertions and removals.
- Testing map put, get, remove, keySet, values, and entrySet.
- Testing hash table collisions and load-factor calculation.
- Including a written or printed example that explains AVL rotations and red-black tree properties conceptually.
- Testing BST insertion, search, traversal, height, and deletion cases.
- Testing comparison examples using Java Collections Framework classes.
- Testing edge cases such as empty structures, one-element structures, duplicate keys, missing keys, and removal from an empty structure.

14. Deliverables

Each group must submit a ZIP file on Brightspace containing:

- Task.java and StudentRecord.java
- Entry.java
- PriorityQueue.java and at least one list-based priority queue implementation
- HeapPriorityQueue.java
- Map.java and UnsortedTableMap.java or another clearly documented map implementation
- HashTableMap.java using separate chaining or linear probing

- A short balanced search tree concepts section in the report, covering AVL trees and red-black tree properties
- BinarySearchTree.java and any required Node class
- Main.java
- A short lab report in PDF format

The report should include:

- Team members names and student numbers
- Description of the implementation and design choices
- Screenshots or copied output from test runs
- Complexity analysis table
- Answers to the required written questions
- Short discussion comparing priority queues, heaps, maps, hash tables, BSTs, balanced search tree concepts, and Java Collections Framework classes
- Problems encountered and how they were solved

15. Laboratory Demonstration Requirement

The laboratory demonstration is a mandatory component of Lab 3. Each team must demonstrate its implementation to the Teaching Assistant (TA).

- Each team must complete a demonstration with the TA for the lab report to be considered for grading.
- All team members must be present during the demonstration.
- Each member may be asked to explain the implementation, tests, hash table behavior, heap operations, BST deletion, balanced search tree concepts, and complexity analysis.
- The demonstration should clearly show that the submitted code runs correctly and matches the requirements of this lab.
- If a team does not complete the laboratory demonstration with the TA, the maximum mark the team can receive for the lab is 50%. This maximum would only apply to an exceptionally strong and complete written report. In general, without the required demonstration, the final lab grade is expected to be significantly lower than 50%.

16. Grading Rubric

Total: 100 points

Component	Points
Priority Queue ADT and list-based implementation	15
Heap-based priority queue implementation	18
Map ADT implementation and key-value operations	12
Hash table implementation, collision handling, and load factor	18
Binary search tree implementation, traversal/deletion, and balanced-tree concepts	17
Java Collections Framework comparison	6
Testing, edge cases, and demonstration quality	7
Complexity analysis and written justification questions	5
Code quality, comments, organization, and report quality	2
Total	**100**

17. Important Notes

- Late submissions are not accepted unless an official extension has been granted by the professor.
- All team members must understand the submitted code.
- Students may be asked to explain their implementation during the lab or tutorial.
- Code must be written in Java.
- Plagiarism or copied code will receive a grade of zero and may be reported according to university academic integrity policies.
- Students may use the Java Collections Framework only in Part F for comparison, unless explicitly stated otherwise. The required custom structures must be implemented by the students.

18. Timeline

Date	Work
Monday, June 1	Lab statement becomes available. Read the full statement and review priority queues, heaps, maps, hash tables, and BSTs.
June 2-5	Create the Java project structure. Begin Task.java, StudentRecord.java, Entry.java, and PriorityQueue.java.
June 6-9	Implement a list-based priority queue and test insert, min, and removeMin operations.
June 10-13	Implement HeapPriorityQueue.java. Test upheap, downheap, and internal heap-array behavior.
June 14-16	Implement Map.java and an unsorted table map. Begin Java Collections Framework comparison.
June 17-20	Implement the hash table using separate chaining or linear probing. Test collisions and load-factor behavior.
June 21-23	Implement the binary search tree. Test insertion, search, traversals, height, and deletion cases. Begin the AVL tree and red-black tree concepts discussion.
June 24	Complete Java Collections Framework comparison, finalize the balanced search tree concepts section, and finalize complexity analysis.
June 25	Complete written justification questions and prepare output/screenshots.
June 26	Complete final testing, prepare the report, and ensure all files are included in the ZIP file.
Saturday, June 27	Submit the ZIP file and laboratory report on Brightspace by 11:59 PM.